

The sheet you actually have

Roadmap review — L5R 4e character sheet

The ten-phase roadmap is well-judged about *value* and consistently wrong about *architecture*. It plans work for a component-based application. The sheet is one HTML file with one script tag, and roughly a third of what the roadmap proposes to build is already built and load-bearing.

10,445	lines in one file, one <code><script></code> , one IIFE, zero external scripts
57	exact-text splice anchors across Features 6–8 that a UI refactor would break
979	regression assertions across 10 suites, currently all green
0	schema version fields in the save, which Phase 7 assumes exists

Every phase's "Engineering Scope" names components, props and reactive bindings. There is no framework here — the only `React` in the file is the phrase "Reactions Stage" from the combat rules.

Already built

Verified by `grep` against the Feature 8 build.

Five roadmap items propose introducing machinery that exists, ships, and has tests standing on it. These aren't near-misses — `makeRollContext` is the spine of six features.

Phase proposes	Already exists as	Consequence
P3 — "Introduce a RollContext object"	<code>makeRollContext()</code> , <code>PREROLL_MODIFIER_REGISTRY</code> , <code>getPreRollModifiers</code> , <code>applyPreRollModifiers</code> , <code>rollWithModifiers</code>	Six features already register contributors into it. Introducing a second one would fork the roll pipeline.
P3 — "Move Void-spend logic into RollPreview"	Feature 4: <code>VOID_SPEND_LIBRARY</code> , one-roll effects consumed inside <code>rollWithModifiers</code>	96 assertions guard this. It is a UI relocation, not new logic — and relocation is the risky half.
P4 — "Explain This Roll"	<code>attachRollModifierBreakdown()</code> renders an itemised bar; <code>getWeaponDamageDice()</code> returns <code>breakdown[]</code> and <code>debugExplanation</code>	The data is computed and displayed today. Phase 4 is a presentation upgrade, not an engine.
P7 — "Add JSON snapshot export/import"	<code>collectData()</code> / <code>applyData()</code> , with a 42-field invariant and save->load->save stability tests	Export/import is done. The genuinely new work in Phase 7 is the audit log and migrations.
P8 — "Why can't I cast this?"	<code>spellEligibility()</code> , plus <code>hasSpellScroll()</code> and the memorisation flags	The predicate exists; the work is returning <i>reasons</i> rather than a boolean.

Five risks the roadmap doesn't name

Ordered by what would hurt most.

01 — Collapsible cards are a data-loss vector · CRITICAL

Phase 1 is rated *low risk*. It is the highest-risk phase in the document. `collectData()` builds the save by querying the DOM — `#techList .entry`, `#equipBody tr`, `#weaponsBody tr`. If a collapsed card unmounts its content rather than hiding it, **the next save silently writes empty lists**. The character is gone and nothing errors.

The harnesses are coupled the same way — 12 direct `#weaponsBody` references and 10 `.entry` references — so they'd break too, which is the lucky part: the gate would catch it before a player did.

Constraint to write into the phase: collapse is `display:none` or `max-height`, never removal. Add an assertion that `collectData()` is identical collapsed and expanded.

02 — A UI refactor breaks the build system · CRITICAL

Features 6, 7 and 8 aren't source files — they're Python splice scripts that match **57 exact runs of source text** and refuse to run if any anchor doesn't appear exactly once. That's deliberate and it has caught real mistakes.

But it means reformatting CSS or restructuring markup invalidates anchors across the whole chain, and the rebuild stops working. Phase 1 and Phase 2 both do exactly that.

Needs a Phase 0: collapse the splice chain into one canonical baseline file, so later phases edit a file rather than a stack of patches.

03 — Phase 7 assumes a schema version that doesn't exist · HIGH

The `VersionManager` and its migrations need to know what version a save is. There is no version field — `grep` returns zero. Adding one is itself a save-shape change, and the project has a hard **42-field invariant** asserted in six suites.

The audit log has the same homelessness problem: an append-only log cannot live in the 42-field save without breaking that invariant. Decide up front — separate `localStorage` key, separate export, or a deliberate versioned bump — because retrofitting it is much worse.

04 — The dependency chain buries the best work · HIGH

Phase 3 is the roadmap's own "very high benefit" item and its first real gameplay milestone. It sits behind Phase 2 (sidebar), which sits behind Phase 1 (collapsible cards, scroll-to-top).

Phase 3's actual dependency on either is close to zero — it needs `RollContext`, which already exists. The stated dependencies are "Void pips in the sidebar" and "colour-coded affinity indicators", both of which are presentation the preview can render itself.

05 — "Do not modify production code" inverts the project's discipline · MODERATE

Eight features have shipped the same way: splice script -> build -> harness -> regression gate. The tests are written *with* the change and are what make it safe to land.

Characterization tests for existing behaviour are worth having. But as the per-phase deliverable they produce tests that nothing is obliged to keep passing. Every phase should end with **"its suite joins the regression gate"** — the number that matters is that 979 stays green and grows.

Phase by phase

Effort re-estimated against what's in the build.

#	Phase	Verdict	Notes	Effort
1	UI/UX foundations	Risk mis-rated	Colour-coding is nearly free — <code>--air-color</code> , <code>--earth-color</code> , <code>--fire-color</code> , <code>--water-color</code> and <code>--void-slot-color</code> already exist and already drive the spell-slot pips. Scroll-to-top is trivial. Collapsible cards carry the data-loss risk above.	low effort · risk low critical
2	Quick-access sidebar	Sound, mis-scoped	Genuinely useful in play. But "reactive bindings to character state" describes a framework that isn't here — the mechanism available is <code>recalcAll()</code> , which already runs on every input event. The sidebar should be another thing <i>it</i> repaints.	moderate · watch recalc cost, not re-render loops
3	Smart roll preview	Half built	The highest-value item, and cheaper than rated: <code>RollContext</code> and the modifier registry exist. The new work is a pre-roll render of what the pipeline already computes post-roll, plus TN display. Moving Void spending is the risky part.	moderate low–moderate
4	Explain this roll	Mostly built	The breakdown already exists and already renders — Part B returns a <code>breakdown[]</code> per damage roll and P2 paints an itemised modifier bar. This is the same object as Phase 3 viewed twice , which is why the two should merge.	moderate low
5	Character creation linting	Sound	High value and well-judged. Much of the rules knowledge is already present — <code>hasSchoolSkillOverlap</code> , <code>spellEligibility</code> , XP tracking, ring caps — so the work is largely surfacing checks that exist as a report, plus the genuinely new rules.	moderate · risk: RAW accuracy, as stated
6	Kata / technique synergy	Most speculative	The one phase where the rules genuinely aren't in the build. Techniques are free text; a synergy engine has to parse them. Feature 7's technique-waiver scan is the precedent, and it works only because it matches four narrow phrasings and has a manual escape hatch.	high · risk: false positives, as stated

#	Phase	Verdict	Notes	Effort
7	Data integrity & persistence	Split it	Three items of very different cost. Export/import already exists . Versioning is moderate but needs the schema field decided first. The audit log is the expensive one and has nowhere to live yet.	export low · versioning moderate · audit log high
8	"Why can't I cast this?"	Cheaper than rated	<code>spellEligibility()</code> already decides castability; the diagnostic is that function returning its reasons instead of a verdict. Listed as depending on the synergy engine — it doesn't, and shouldn't wait for it.	^{high} moderate
9	Polish & immersion	Well-judged	Correctly rated. The sheet is already fully tokenised — <code>--paper</code> , <code>--ink</code> , <code>--shu</code> , <code>--gold</code> , <code>--line</code> — so clan skins are a palette swap rather than a restyle. Cheapest visible win in the document.	low · risk low
10	Future expansions	Defer, correctly	Right to defer. Worth noting that equipment automation is partly begun: armour already feeds <code>f_currentTN</code> , and Features 7 and 8 both add terms to that same sum. Weapon traits are the untouched half.	high · deferral stands

A resequenced order

Same ten phases, ordered by value delivered per unit of risk.

Two structural changes: a new Phase 0 that unblocks everything, and the merge of Smart Roll Preview with Explain This Roll — they read the same object and splitting them means building the same render twice.

New #	Phase	Was	Why here
0	Baseline consolidation	<i>new</i>	Collapse the splice chain into one canonical file; freeze the harness set; add a schema version to the save while the save shape is already being touched. Unblocks every phase that edits markup.
1	Roll preview + explain this roll	3 + 4	Merged. Highest value, and the engine is already there — this renders pre-roll what the pipeline computes post-roll. Do the read-only preview first; move Void spending after it works.
2	Character creation linting	5	No UI dependency, high value, most rules already present. Its <code>ValidationReport</code> is also the natural home for Phase 8's casting diagnostics later.

New #	Phase	Was	Why here
3	UI foundations	1	Now safe: the baseline exists, so anchors aren't at stake. Collapse via CSS only, with the <code>collectData()</code> equivalence assertion written first.
4	Quick-access sidebar	2	Genuinely depends on the UI foundation, which it now has. Repaint from <code>recalcAll()</code> .
5	Casting diagnostics	8	Extends the linting engine from step 2 rather than waiting on the synergy engine it was listed under.
6	Data integrity	7	Versioning and audit log, now that the schema version landed in Phase 0. Export/import is already done.
7	Synergy engine	6	Highest rules risk and most speculative. Benefits from every layer above, and is the one phase that can be dropped without stranding anything.
8	Polish & theming	9	Cheap, visible, and pleasant to do after the heavy phases.
9	Future expansions	10	Unchanged.

Rewriting the harness prompt

The per-phase prompt is the same nine lines each time, and two of them work against this codebase. Suggested replacement, to be pasted per phase:

- **Drop "do not modify production code."** Replace with: the feature and its suite ship together, produced by a splice script that reproduces the build byte-for-byte.
- **Add the regression gate explicitly.** "All existing suites must stay green — currently 10 suites, 979 assertions — and the new suite joins them."
- **Add the save-shape invariant.** "`data.fields` stays at exactly 42 entries unless the phase's brief says otherwise, in which case say so and bump the schema version."
- **Add a dormancy clause.** Every feature since Feature 6 has been dormant until explicitly activated; it's the single reason the regression suites survived eight features.
- **Name the frozen functions.** `rollDicePool`, `rollWeaponDicePool`, `rollExplodingD10`, `applyTenDiceRule`, `showRollResult`.

Findings verified against `l5r-character-sheet part C feature 8 mirumoto.html` — 10,445 lines, 1,191,524 characters — and the splice scripts and harnesses shipped alongside it. Line, anchor and assertion counts are measured, not estimated.